Lab 9

Task 1:

Code	Explanation

[org 0x0100]
jmp start
and_result: db 0
or_result: db 0
invert_lower_bits_result: db 0
start:
; clearing the lower bit
mov al, 9Ah
and al, F0h
mov [and_result], al
 	
;setting the lower bit
mov al, 9Ah
or al, 0Fh
mov [or_result], al
 	
; inverting the lower bit
mov al, 9Ah
xor al, 0Fh
mov [invert_lower_bits_result], al
 	
mov ah, 4ch
int 21h
	➢ Line 1: start of the program
	➢ Line 2: Jump to start label
	➢ Line 3: initialized label 'and_result' to store result after and operation
	➢ Line 4: initialized label 'or_result' to store result after or operation.
	➢ Line 5: initialized label 'invert_lower_bits_result' to store result after xor operation
	➢ Line 6: label 6.
	➢ Line 7: Storing 9Ah (1001 1010d) in al register.
	➢ Performing AND operation by taking mask bit (1111 0000d).
	➢ Moving result in 'and_result' the result will be now (1001 0000).
	➢ Same or operation using bit mask (0000 1111d) ans store the result in or_result: (1001 1111d)
	➢ Same xor operation using bit mask (0000 1111d) and store the result in invert_lower_bit_result: (1001 0101d).

Task 2:

Code	Explanation

[org 0x0100]
jmp start
bit_flag: db 0
start:
mov ax, 0000h
 	
test ax, 00001000b
jz bit_not_set
mov byte [bit_flag], 1
jmp bit_checked
bit_not_set:
mov byte [bit_flag], 0
bit_checked:
mov ah, 4ch
int 21h
	• Initializing label bit_flag to 0.
	• Moving value 0 to ax.
	• Testing if the 3rd bit of ax register is 1 or not using mask bit (00001000b)
	• Jump to bit_not_set if result is zero and set bit_flag to 0.
	• Else set bit_flag to 1.
	• Terminate the program.

Task 3:

Code	Explanation

[org 0x0100]
jmp start
masked: db 0
clear_lower:
  and bl, F0h
  ret
start:
  mov bl, 9Ah
  call clear_lower
  mov [masked], bl
mov ah, 4ch
int 21h
	➢ Initialized label masked to zero.
	➢ Jump to start label.
	➢ Move value 9Ah (1001 1010d) into bl register.
	➢ Call subroutine clear_lower
	➢ Perform and operation between bl and bit mask F0 (1111 0000d).
	➢ Store the result in masked label.
	➢ Terminate the program.

Task 4:

Code	Explanation

[org 0x0100]
jmp start
arr: dw 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
sum: dw 0
sum_array:
  push bx
  push cx
 	
  xor ax, ax
  mov bx, arr
  mov cx, 10
  sum_loop:
  add aw, [bx]
  add bx, 2
  dec cx
  jnz sum_loop
  mov [sum], ax
  pop cx
  pop bx
  ret
 	
start:
  call sum_array
  mov ah, 4ch
  int 21h
	➢ Initializing arr label to 10 values.
	➢ Jumping to start label.
	➢ Call the subroutine sum_array:
	➢ Push (save current value of bx in the stack for future use).
	➢ push/save current value of cx in the stack.
	➢ Clearing ax register.
	➢ Mov first value of arr in bx.
	➢ Move 10 in cx register for running the loop.
	➢ Start of loop.
	➢ Adding bx and ax values.
	➢ Adding 2 into bx so that we can move to next number stored in arr.
	➢ Decrease the counter by 1.
	➢ Move the result in sum label.
	➢ Pop cx. Restore the saved cx register from the stack.
	➢ Pop bx restore the saved bx register value from the stack.
	➢ Return from the function
	➢ Terminate program

Task 5:

Code	Explanation

[org 0x0100]
jmp start
ax_after: dw 0
bx_after: dw 0
start:
  mov ax, 1111h
  mov bx, 2222h
 	
  push ax
  push bx
 	
  pop ax
  pop bx
 	
  mov [ax_after], ax
  mov [bx_after], bx
mov ah, 4ch
int 21h
	➢ Jumping to start label
	➢ Initialized some labels to 0. Like ax_after and bx_after.
	➢ Moving 1111h in ax and 2222h in bx register.
	➢ Push ax and bx in stack (1111 2222).
	➢ Pop ax (2222h)
	➢ Pop bx (1111h).
	➢ Storing the result of ax and bx in respective labels
	➢ Terminating the program.